

Design and Implementation of an INT8 GEMM Accelerator for Quantized MNIST Inference

Ruthika Kodiganti

Abstract—This project presents a parameterizable General Matrix Multiplication (GEMM) accelerator implemented in Verilog RTL. The accelerator employs an 8-way parallel INT8 multiply-accumulate (MAC) architecture with INT32 accumulation and is integrated into a quantized MNIST inference pipeline to demonstrate a practical machine learning workload. The complete inference flow is partitioned across three domains. A Python framework trains a neural network, performs post-training quantization, and exports quantized parameters. The GEMM accelerator performs only matrix multiplication and accumulation. A Verilog testbench orchestrates data movement between layers and performs neural network post-processing, including bias addition, ReLU activation, requantization, and prediction generation. The design explores architectural concepts such as parallelism, pipelining, and memory organization while demonstrating the role of GEMM accelerators in neural network inference.

I. INTRODUCTION

Neural network inference is dominated by matrix multiplication operations. In fully connected layers, neuron activations are computed through matrix-vector products followed by bias addition and activation functions. As model sizes continue to increase, matrix multiplication becomes a primary computational bottleneck, motivating the use of dedicated hardware accelerators.

General Matrix Multiplication (GEMM) forms the computational core of many machine learning workloads and is therefore a common target for hardware acceleration. Efficient GEMM implementations require careful consideration of memory organization, parallelism, and data movement. The goal of this project is to design and verify a configurable GEMM accelerator using synthesizable Verilog RTL. The project also serves as a platform for exploring hardware architecture concepts such as finite state machine control, parallel MAC execution, pipelined computation, and SRAM-based memory systems. To demonstrate practical applicability, the accelerator is integrated into a complete quantized MNIST inference pipeline.

II. NEURAL NETWORK OVERVIEW

The accelerator is evaluated using a fully connected neural network trained on the MNIST handwritten digit classification dataset.

The network architecture consists of:

- Input Layer: 784 neurons
- Fully Connected Layer 1: $784 \rightarrow 32$
- ReLU Activation
- Fully Connected Layer 2: $32 \rightarrow 10$

The output layer produces logits corresponding to the 10 MNIST digit classes. The network is trained in PyTorch using the Adam optimizer and cross-entropy loss.

III. SYSTEM OVERVIEW

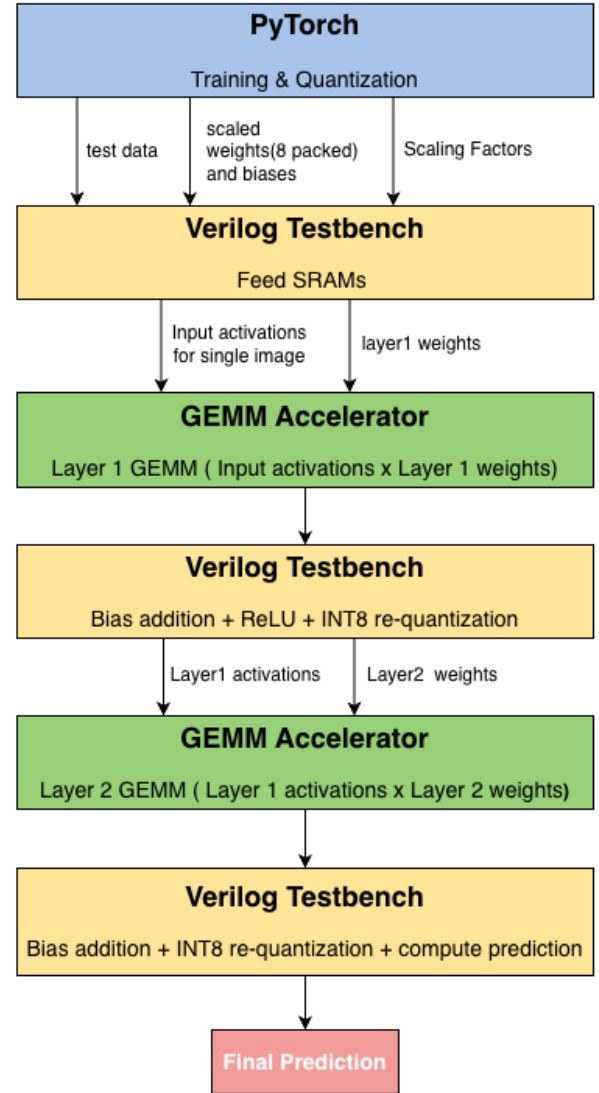


Fig. 1. System Overview

The complete inference flow consists of three major components: a Python framework, a Verilog testbench, and the

GEMM accelerator. The Python framework is responsible for neural network training, quantization, and parameter export. The Verilog testbench acts as a simplified host processor that loads data into memory, launches accelerator execution, performs neural network post-processing, and evaluates classification accuracy. The GEMM accelerator serves as the compute engine and performs matrix multiplication operations.

A. Python Framework

A two-layer fully connected neural network is trained on the MNIST dataset using PyTorch. Following training, activation statistics are collected to determine quantization scales. Weights and activations are quantized to INT8 values, while biases are quantized to INT32 values. Scaling factors are stored in Q16.16 fixed-point format to enable fixed-point requantization during inference.

The quantized parameters are exported into hardware-readable memory files. Weights are packed into an 8-lane format that matches the parallel MAC architecture of the accelerator. Test images, quantized biases, and scaling factors are also exported for use during RTL simulation.

B. GEMM Accelerator

The accelerator is responsible solely for matrix multiplication and accumulation. Given matrices A and B , it computes:

$$C = A \times B \quad (1)$$

using INT8 operands and INT32 accumulators. Neural network specific operations such as bias addition, activation functions, and requantization are intentionally excluded from the hardware datapath.

C. Verilog Testbench

The Verilog testbench coordinates execution of the complete inference flow. Exported weights and activations are loaded into SRAM models using `$readmemh`. The testbench configures matrix dimensions, launches accelerator execution, and waits for completion.

After Layer 1 matrix multiplication, the testbench performs bias addition, ReLU activation, and requantization before writing the resulting activations back into SRAM. These activations then become the input to Layer 2. Following Layer 2 execution, the testbench performs bias addition and requantization before selecting the output neuron with the highest value as the final prediction.

D. Separation of Responsibilities

The accelerator is intentionally restricted to matrix multiplication. This design choice improves modularity and allows the hardware to be reused for applications beyond neural networks. Furthermore, activation functions, quantization strategies, and post-processing operations frequently vary between models. By keeping these operations in software, the hardware datapath remains simple while retaining flexibility for different workloads.

IV. GEMM ACCELERATOR ARCHITECTURE

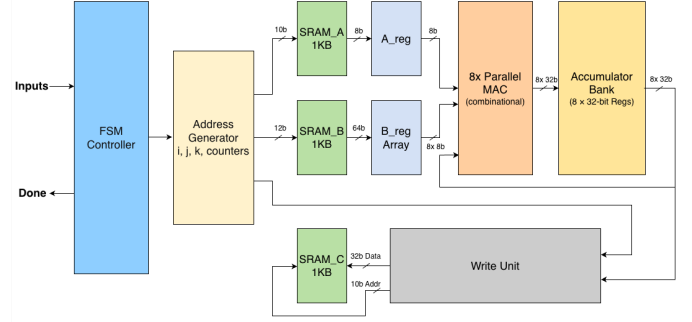


Fig. 2. GEMM Architecture

The accelerator consists of the following components:

- FSM Controller
- Address Generator
- SRAM_A
- SRAM_B
- SRAM_C
- Input A Register
- Input B Register Array
- 8-Way Parallel MAC Array
- Accumulator Bank
- Write Unit

The FSM controller coordinates memory accesses, computation, and writeback operations. The address generator produces memory addresses based on loop indices and matrix dimensions. The MAC array performs the parallel multiply-accumulate operations, while the accumulator bank stores intermediate results before they are written to SRAM_C.

A. Inputs and Outputs

Inputs:

- start
- Matrix dimensions M, K, and N
- Base addresses for matrices A, B, and C

Outputs:

- done

The matrix dimensions are programmable at runtime, allowing the same accelerator to execute different GEMM workloads.

B. FSM Operation

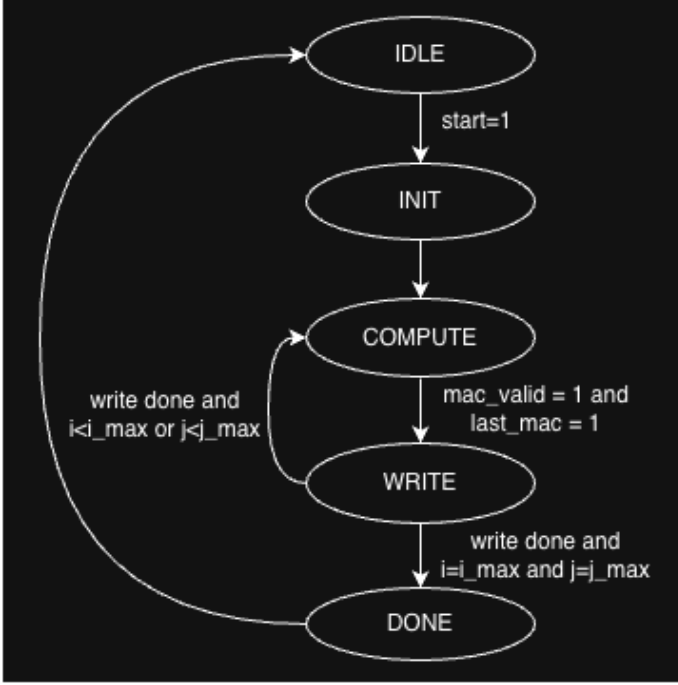


Fig. 3. FSM State Diagram

The accelerator is controlled by a finite state machine (FSM) consisting of five states: IDLE, INIT, COMPUTE, WRITE, and DONE. This controller coordinates memory accesses, computation, and output writeback while ensuring correct sequencing of operations.

In the IDLE state, the accelerator waits for a start pulse from the host. Matrix dimensions and base addresses are captured and stored in internal registers. Once a valid start signal is received, the accelerator transitions to the INIT state.

The INIT state initializes loop counters, accumulator registers, pipeline control signals, and boundary conditions required for computation. Matrix dimensions are used to compute the maximum values of the loop counters, allowing the accelerator to support configurable matrix sizes.

The COMPUTE state performs the core GEMM operation. Memory addresses are generated for matrices A and B, activations and weights are fetched from SRAM, and the MAC array performs multiply-accumulate operations. Computation proceeds in a pipelined manner until all values corresponding to a particular output block have been accumulated.

Once accumulation is complete, the accelerator enters the WRITE state. Accumulated results are written to SRAM_C one output element at a time. After all valid outputs have been stored, the controller either returns to COMPUTE to process the next output block or transitions to DONE if all matrix elements have been computed.

The DONE state asserts the done signal, indicating completion of the GEMM operation. The controller then returns to IDLE and waits for the next request.

C. Computation Model

The GEMM accelerator computes matrix multiplication using a three-level nested loop structure. The row index i selects a row of matrix A, the column-block index j selects a group of N_MAC output columns, and the reduction index k iterates through the shared dimension of the multiplication.

The equivalent algorithm is shown below.

```

for i = 0 to M-1
  for j = 0 to ceil(N/N_MAC)-1
    accum[0:N_MAC-1] = 0
    for k = 0 to K-1
      a_in = A[A_base + i*K + k]
      b_in[0:N_MAC-1] =
        B[B_base + j*K + k]
      for lane = 0 to N_MAC-1
        accum[lane] =
          accum[lane] +
            a_in * b_in[lane]
      for lane = 0 to N_MAC-1
        if (j*N_MAC + lane < N)
          C[C_base + i*N +
            j*N_MAC + lane]
            = accum[lane]
  
```

During each iteration of the k loop, a single activation value is read from SRAM_A and broadcast to all MAC units. Simultaneously, a packed 64-bit word is read from SRAM_B, supplying one INT8 weight to each of the 8 MAC lanes. The MAC array therefore computes 8 partial products in parallel every cycle.

Mathematically, each MAC lane performs

$$accum[lane] = accum[lane] + A[i][k] \times B[k][j \cdot N_MAC + lane] \quad (2)$$

for $lane = 0 \dots N_MAC - 1$.

After all K iterations are complete, the accumulated values correspond to the output elements

$$C[i][j \cdot N_MAC : j \cdot N_MAC + N_MAC - 1] \quad (3)$$

and are written back to SRAM_C during the WRITE state. Boundary checking is performed to prevent invalid writes when the number of output columns N is not an exact multiple of N_MAC .

V. DESIGN CHOICES

A. Parallelism

The accelerator employs eight parallel MAC units, enabling eight output columns to be computed simultaneously. This level of parallelism was selected as a compromise between throughput and implementation complexity. Increasing the number of MAC units would improve performance but would also require additional memory bandwidth, larger register structures, and more complex routing.

Matrix A is broadcast to all MAC units rather than being tiled. During GEMM computation, a single activation value is reused across multiple output columns, making broadcasting a natural and bandwidth-efficient choice. Matrix B , in contrast, is packed to supply one weight per MAC unit every cycle.

The design accumulates output values locally within the accelerator and writes results sequentially to memory. Although a wider output interface could reduce write latency, the current approach simplifies memory organization and control logic while remaining sufficient for the target workload.

B. Memory Organization

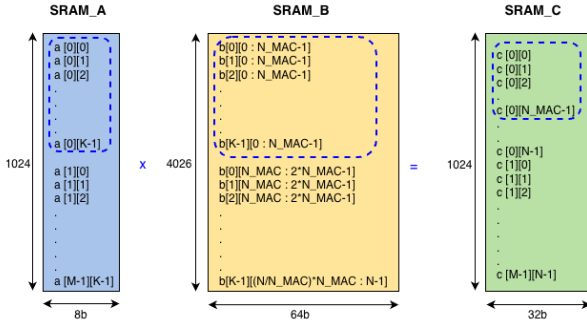


Fig. 4. Memory Organization

Three SRAM blocks are used to store input activations, weights, and output accumulations.

SRAM_A stores INT8 activation values. For the MNIST workload, the input image or intermediate layer activations are loaded into this memory before accelerator execution begins.

SRAM_B stores INT8 weights using a custom packed layout optimized for the 8-way parallel MAC architecture. Instead of storing weights in a conventional row-major format, eight weights corresponding to eight output neurons are packed into a single 64-bit memory word. As a result, a single memory access supplies one weight to each MAC unit, maximizing utilization of the parallel compute resources.

SRAM_C stores INT32 accumulated outputs. Since multiple INT8 products are accumulated together, a wider storage format is required to avoid overflow and preserve numerical accuracy.

A key design choice is the storage of complete matrices inside SRAM prior to execution. While larger accelerators often employ tiled memory systems that continuously stream

data from external memory, such approaches require significantly more complex control logic. For the relatively small MNIST workload, storing complete matrices simplifies the implementation and allows the focus to remain on the accelerator architecture itself.

C. Compute Pipeline

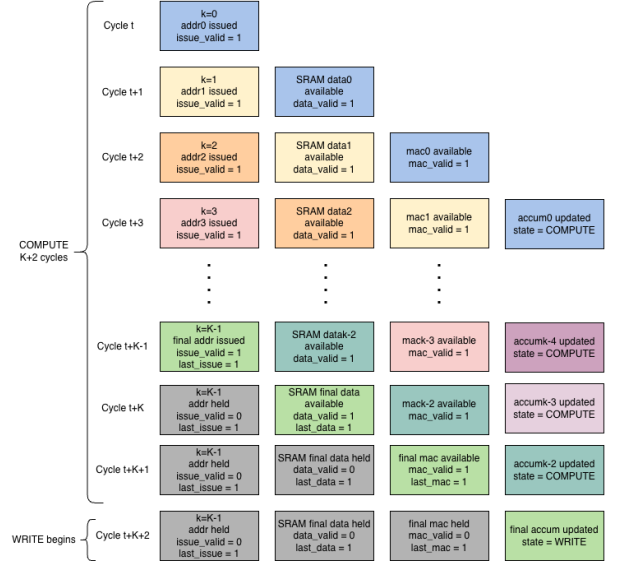


Fig. 5. Compute Pipeline

The COMPUTE state is implemented as a multi-stage pipeline to improve throughput by overlapping memory access and arithmetic operations.

The first stage generates memory addresses for matrices A and B using the current values of the loop counters. These addresses are presented to the SRAM interfaces and initiate memory reads.

During the second stage, the requested activation and weight data become available at the SRAM outputs. The activation value is captured into a dedicated register while the packed weight word is distributed across the eight MAC lanes.

The third stage performs multiplication and accumulation. Each MAC unit computes the product of the shared activation value and its corresponding weight before adding the result to its local accumulator.

The final stage updates the accumulator bank with the newly computed values.

Pipeline progress is tracked using the `issue_valid`, `data_valid`, and `mac_valid` signals. These signals indicate when addresses, memory data, and MAC outputs are valid, respectively. Additional control signals, namely `last_issue`, `last_data`, and `last_mac`, are used to drain the pipeline after the final memory access has been issued.

Without pipelining, the accelerator would need to wait for one multiplication to complete before issuing the next memory request. By overlapping address generation, memory access,

and computation, the design allows new work to enter the datapath every cycle once steady-state execution is reached, significantly improving throughput.

D. Quantization and Scaling

The accelerator uses INT8 activations and weights with INT32 accumulation. This quantization strategy was chosen to reduce memory usage and computational complexity while maintaining acceptable classification accuracy.

Compared to floating-point representations, INT8 values require significantly less storage and enable simpler hardware multipliers. Since matrix multiplication dominates the computational workload, reducing operand precision provides substantial savings in both memory bandwidth and hardware cost.

Quantization is performed offline in Python after model training. Activation scales are determined through a calibration process using representative MNIST samples. The 99.9th percentile magnitude of each activation tensor is used to compute the scaling factor, reducing sensitivity to outliers while preserving the dynamic range of the majority of values.

Weights and activations are quantized to INT8 values, while biases are quantized to INT32 values. Higher precision is required for biases because they are added to the INT32 outputs of the accumulation process.

To avoid floating-point arithmetic during inference, all scaling factors are stored in Q16.16 fixed-point format:

$$S_{fixed} = \text{round}(S \times 2^{16}) \quad (4)$$

After each matrix multiplication stage, the Verilog testbench performs requantization using precomputed fixed-point scaling multipliers. This converts the INT32 accumulated outputs into the INT8 range expected by the next layer while avoiding floating-point operations.

By performing quantization and scaling in software, the accelerator datapath remains focused on matrix multiplication, allowing a simpler and more reusable hardware design.

VI. RESULTS

The proposed accelerator was evaluated on the MNIST test set containing 10,000 images. Accuracy was measured at three stages of the inference flow: floating-point PyTorch inference, Python INT8 simulation, and RTL simulation using the GEMM accelerator.

A. Accuracy

Table I compares the classification accuracy obtained from the three implementations.

TABLE I
ACCURACY COMPARISON

Implementation	Accuracy (%)
PyTorch FP32	95.77
Python INT8 Simulation	95.78
RTL Simulation	95.78

The quantized implementation achieves virtually identical accuracy to the original floating-point model. Furthermore, the RTL simulation produces exactly the same predictions as the Python INT8 reference implementation, confirming the correctness of both the quantization flow and the hardware design.

B. Performance

Average cycle counts were collected across the complete MNIST test set.

TABLE II
AVERAGE CYCLE COUNTS PER INFERENCE

Operation	Cycles
Layer 1 GEMM	3178
Layer 2 GEMM	86
Total Inference	3264

Layer 1 dominates the execution time due to its larger matrix dimensions $(1 \times 784) \times (784 \times 32)$, while Layer 2 performs a significantly smaller computation of size $(1 \times 32) \times (32 \times 10)$.

C. MAC Utilization

The effectiveness of the 8-way parallel MAC architecture was evaluated by comparing the number of useful MAC operations with the total available MAC execution slots in the measured cycle counts.

TABLE III
MAC ARRAY UTILIZATION

Metric	Value
Layer 1 Utilization	98.68%
Layer 2 Utilization	46.51%
Overall Utilization	97.30%

Layer 1 achieves near-ideal utilization because its output dimension (32 neurons) is an exact multiple of the accelerator parallelism ($N_{MAC} = 8$). Layer 2 exhibits lower utilization because the output dimension (10 neurons) requires a partially filled final output block, leaving several MAC lanes idle.

Despite this effect, the overall utilization remains above 97%, demonstrating that the chosen 8-way architecture is well matched to the target workload.

D. Summary

The accelerator successfully executes quantized neural network inference while preserving the accuracy of the original floating-point model. Identical results between the Python INT8 simulation and RTL implementation validate the correctness of the hardware design, while the measured cycle counts and MAC utilization demonstrate efficient use of the parallel compute resources.

VII. FUTURE WORK

Future improvements include tiled matrix processing, FPGA deployment, host processor interfaces and systolic-array based architectures.

VIII. CONCLUSION

This project presented the design and verification of a parameterizable INT8 GEMM accelerator implemented in Verilog RTL. The accelerator employs an 8-way parallel MAC architecture, pipelined execution, and SRAM-based memory organization to efficiently execute matrix multiplication workloads.

The design was successfully integrated into a quantized MNIST inference pipeline, demonstrating end-to-end hardware accelerated inference and providing insight into the architectural principles underlying modern AI hardware accelerators.